

Hexlet AI - Проект №1: "Календарь" (Задание)

Обновлено 10 июня 2026, 23:29 Михаил Иконников

Содержание страницы

Календарь звонков

[Чему учимся](#)[Описание проекта](#)[Демонстрация](#)

1. Создание репозитория на GitHub

[Чему учимся](#)[Ссылки](#)[Задачи](#)[Результат шага](#)

2. Проектирование приложения

[Описание приложения](#)[Ссылки](#)[Задачи](#)[Результат шага](#)

3. Фронтенд

[Требования](#)[Ссылки](#)[Задачи](#)[Результат](#)

4. Реализация бэкенда

[Требования](#)[Задачи](#)[Результат](#)

5. Интеграционные сценарии (Playwright)

[Требования](#)[Ссылки](#)[Задачи](#)[Результат](#)

6. Docker и деплой

[Требования](#)[Ссылки](#)[Задачи](#)[Результат](#)

7. Как мы проверяем проект

[Что должно работать](#)[Требования к оформлению](#)[Что дальше](#)

Календарь звонков

Проект модуля #1: <https://ru.hexlet.io/programs/ai-for-developers/projects/386>

Разработайте совместно с ИИ сервис для бронирования календаря

- Срок: 2-4 недели

Чему учимся

В этом проекте мы проходим полный цикл разработки небольшого веб-приложения с ИИ как рабочим инструментом. Сначала нужно определить, что именно должно делать приложение, и зафиксировать контракт между клиентом и сервером. Затем на основе этого контракта последовательно собрать интерфейс, реализовать бэкенд, покрыть основные сценарии тестами и подготовить сервис к деплою.

Проект учит не просто писать код, а выстраивать разработку по этапам. Здесь важно сначала договориться о поведении системы, потом реализовать его в интерфейсе и на сервере, а в конце проверить все на уровне пользовательских сценариев.

Ключевой формат работы в этом проекте: всю разработку ведем с помощью ИИ-агентов. В идеале вы не пишете вручную ни одной строки кода: формулируете задачу агенту, проверяете результат и итеративно улучшаете решение.

Проект можно выполнить на любом стеке технологий. Ограничение задается не языком или фреймворком, а итоговым результатом: приложение должно собираться в Docker-образ и запускаться в контейнере. Такой формат позволяет сосредоточиться на архитектуре и контракте API, а не на выборе конкретного инструментария.

Описание проекта

В проекте нет авторизации, личных кабинетов и интеграций с внешними календарями. Функциональность намеренно ограничена, чтобы сосредоточиться на главном.

Демонстрация

Calendar

ЗаписатьсяАдминка

TotaHost

Выберите тип события

Нажмите на карточку, чтобы открыть календарь и выбрать удобный слот.

Встреча 15 минут15 мин

Короткий тип события для быстрого слота.

Встреча 30 минут30 мин

Базовый тип события для бронирования.

Calendar

ЗаписатьсяАдминка

Встреча 15 минут15 мин

Короткий тип события для быстрого слота.

Выбранная дата
вторник, 31 марта

Выбранное время
Время не выбрано

Календарь

март 2026 г.

Пн	Вт	Ср	Чт	Пт	Сб	Вс
23	24	25	26	27	28	1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30	31 33 св.	1 35 св.	2 36 св.	3 36 св.	4 36 св.	5 36 св.

Статус слотов

09:00 - 09:15Занято

09:15 - 09:30Занято

09:30 - 09:45Занято

09:45 - 10:00Свободно

10:00 - 10:15Свободно

10:15 - 10:30Свободно

10:30 - 10:45Свободно

НазадПродолжить

1. Создание репозитория на GitHub

1. Подключение к GitHub

Код проекта будет храниться в репозитории на вашем компьютере и на GitHub. Хекслет автоматически создаст и настроит для вас репозиторий, для этого потребуются права на доступ к вашему GitHub-аккаунту. Хекслет будет управлять только репозиториями проектов курса. Ваши приватные репозитории и персональные данные полностью конфиденциальны.

Хекслет создаст репозиторий с названием `ai-for-developers-project-386` в вашем аккаунте на GitHub. Название репозитория важно для правильной работы автоматических проверок — не меняйте его. Если случайно удалите или отредактируете служебные файлы в директории `.github/workflows`, снова нажмите на кнопку «Подготовить репозиторий» для их восстановления. Написанный вами код, который хранится в других директориях, при восстановлении сохранится.

В этом проекте мы проходим полный цикл разработки небольшого веб-приложения с ИИ как рабочим инструментом. Сначала нужно определить, что именно должно делать приложение, и зафиксировать контракт между клиентом и сервером. Затем на основе этого контракта последовательно собрать интерфейс, реализовать бэкэнд, покрыть основные сценарии тестами и подготовить сервис к деплою.

Мы работаем в подходе Design First: сначала фиксируем API-контракт, а затем по нему отдельно реализуем фронтенд и бэкэнд. Для ИИ-агентов это критично: вместо анализа чужой части кода они опираются на явную спецификацию, что снижает расход токенов и требования к модели. Особенно это заметно при изменениях: достаточно обновить контракт и продолжить реализацию без лишнего разбора всей системы.

Проект учит не просто писать код, а выстраивать разработку по этапам. Здесь важно сначала договориться о поведении системы, потом реализовать его в интерфейсе и на сервере, а в конце проверить все на уровне пользовательских сценариев.

Ключевой формат работы в этом проекте: всю разработку ведем с помощью ИИ-агентов. В идеале вы не пишете вручную ни одной строки кода: формулируете задачу агенту, проверяете результат и итеративно улучшаете решение.

Проект можно выполнить на любом стеке технологий. Ограничение задается не языком или фреймворком, а итоговым результатом: приложение должно собираться в Docker-образ и запускаться в контейнере. Такой формат позволяет сосредоточиться на архитектуре и контракте API, а не на выборе конкретного инструментария.

Описание проекта

«Запись на звонок» — это упрощенный сервис бронирования времени по мотивам [Cal.com](#).

The screenshot displays the Cal.com web application interface. On the left, a sidebar identifies the user as Kirill Mokevnin and details a "30 Min Meeting" scheduled for 30 minutes via Google Meet in the Asia/Yekaterinburg time zone. The central area features a monthly calendar for April 2026, where the 7th of the month is highlighted. To the right, a detailed view of the meeting on Tuesday, April 7th, is shown, listing five available time slots: 21:00, 21:30, 22:00, 22:30, and 23:30. The top navigation bar includes a toggle for "Overlay my calendar" and icons for settings, calendar views, and other functions. The Cal.com logo is centered at the bottom.

Cal.com — это сервис, где пользователь публикует доступные интервалы, а другой человек выбирает свободное время и бронирует встречу. У нашего приложения тот же базовый сценарий: владелец публикует доступное время для встреч, а другой пользователь выбирает свободный слот и записывается на звонок.

В проекте нет авторизации, личных кабинетов и интеграций с внешними календарями. Функциональность намеренно ограничена, чтобы сосредоточиться на главном.

В результате должно получиться небольшое, но законченное приложение. Пользователь видит свободные слоты по 30 минут, может выбрать время и оформить запись, а владелец календаря — посмотреть список предстоящих встреч. Этого достаточно, чтобы пройти весь путь от проектирования до запуска сервиса в облаке.

Пример того, что может получиться:

Слайд	Комментарий

Calendar

ЗалісаццаАдмінка

БЫСТРАЯ ЗАПІСЬ НА ЗВОНОК

Calendar

Заброніруйце сустрэчу за хвілінку: выберыце тып падзеі і зручнае час.

Залісацца →

Возможности

- Выбар тыпу падзеі і зручнага часу для сустрэчы.
- Быстрое бронирование с подтверждением и дополнительными заметками.
- Управление типами встреч и просмотр предстоящих записей в админке.

Calendar

ЗалісаццаАдмінка

Tota

Host

Выберите тип события

Нажмите на карточку, чтобы открыть календарь и выбрать удобный слот.

Встреча 15 минут

15 мин

Короткий тип события для быстрого слота.

Встреча 30 минут

30 мин

Базовый тип события для бронирования.

Calendar

Записаться

Админка

Tota

Host

Встреча 15 минут

15 мин

Короткий тип события для быстрого слота.

Выбранная дата

вторник, 31 марта

Выбранное время

Время не выбрано

Календарь

←

→

март 2026 г.

Пн	Вт	Ср	Чт	Пт	Сб	Вс
23	24	25	26	27	28	1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30	31 33 св.	1 35 св.	2 36 св.	3 36 св.	4 36 св.	5 36 св.

Статус слотов

09:00 - 09:15	Занято
09:15 - 09:30	Занято
09:30 - 09:45	Занято
09:45 - 10:00	Свободно
10:00 - 10:15	Свободно
10:15 - 10:30	Свободно
10:30 - 10:45	Свободно

Назад

Продолжить

Ссылки

- [Cal.com](https://cal.com) — референсный сервис, на который можно опираться при проектировании пользовательского сценария.

Задачи

1. Зайдите в [Cal.com](https://cal.com) и зарегистрируйтесь.
2. Изучите возможности сервиса и базовый путь пользователя:
 1. создание типа встречи,
 2. просмотр страницы бронирования
 3. запись на свободный слот.
3. Это нам пригодится для реализации нашего сервиса.

Результат шага

Изучили, как работает [Cal.com](https://cal.com), познакомились с предметной областью и поняли, почему API-контракт нужен как основа для дальнейшей реализации фронтенда и бэкенда с помощью ИИ.

2. Проектирование приложения

- Проект выполняется в подходе **Design First**: сначала фиксируем поведение системы и API-контракт, и только потом переходим к реализации.
- Фронтенд и бэкенд реализуются раздельно и взаимодействуют через API-контракт.
- Контракт в этом шаге задает границу между частями приложения и остается единым источником правды для реализации.
- Спецификация API - это единый источник правды для обеих частей приложения. По ней можно независимо реализовывать фронтенд и бэкенд, не тратя ресурсы на анализ внутренней реализации соседней части.
- Это ускоряет работу агентов, снижает расход токенов и уменьшает требования к модели. При изменениях достаточно обновить контракт и синхронно внести правки в обе части.
- Для этого возьмем TypeSpec - удобный инструмент для создания OpenAPI спецификации.
- Также он позволяет генерировать код практически в любой язык. Он похож на TypeScript, но это лишь визуальное сходство, TypeSpec не полноценный язык программирования.
- Начиная с этого шага мы всю работу выполняем с помощью агентов: описываем план, уточняем детали, реализуем.
- В идеале, мы не должны написать ни одной строчки кода вручную.

Описание приложения

В проекте есть две условные роли: владелец календаря и гости.

При этом в проекте нет регистрации и авторизации.

Владелец календаря — один заранее заданный профиль.

Этот профиль по умолчанию используется в админской части.

Гость бронирует слоты без создания аккаунта и без входа в систему.

Владелец календаря может:

- Создавать типы событий. Для каждого типа события задает id, название, описание и длительность в минутах.
- Просматривает страницу предстоящих встреч, где в одном списке показаны бронирования всех типов событий.

Гость:

- Может посмотреть страницу с видами брони, где доступно название, описание и длительность.
- Выбирает тип события, открывает календарь и выбирает свободный слот в ближайшие 14 дней.
- Создает бронирование на выбранный слот.

Правило занятости:

- На одно и то же время нельзя создать две записи, даже если это разные типы событий.

Окно записи по умолчанию:

- Доступные слоты формируются на 14 дней, начиная с текущей даты.
- Гость может записаться только на свободный слот из этого окна.

Внутреннюю реализацию, стек и структуру проекта вы выбираете сами.

На этом этапе мы договариваемся только о внешнем поведении системы:

- какие сценарии поддерживаются,
- какие данные передаются
- и какие ограничения действуют при бронировании.

Ссылки

- [TypeSpec](#) — инструмент, в котором мы описываем API-контракт и при необходимости генерируем из него OpenAPI-спецификацию.

Задачи

1. Опишите доменные сущности: владелец, тип события, слот, бронирование и публичный сценарий гостя.
2. Сформулируйте задачу агенту и подготовьте [TypeSpec](#) -спецификацию, которая фиксирует [API](#) -контракт.
3. Проверьте, что спецификация покрывает сценарии владельца и гостя.

Результат шага

- В репозитории есть [TypeSpec](#) -спецификация, которая фиксирует поведение приложения для владельца и гостя.

3. Фронтенд

На этом шаге мы делаем UI приложения на основе контракта из предыдущего шага.

Стек фронтенда вы выбираете сами. Здесь нет обязательного языка или фреймворка: важен рабочий интерфейс, который следует контракту. Рекомендуем использовать TypeScript + Vite. Для UI советуем shadcn/ui или Mantine, а для работы с API по контракту во время разработки — Prism. Это рекомендации, а не обязательные требования. Вы можете использовать свои инструменты и стек, если итоговый интерфейс соответствует контракту и требованиям шага.

Почему именно эти технологии: с ними ИИ-ассистенты обычно дают самые точные подсказки и быстрее помогают двигаться по шагам. На таком стеке проще запускать проект, безопаснее работать с типами и легче вносить изменения по мере развития интерфейса.

Если вы раньше не работали с этими инструментами, это нормально. На этом шаге важнее собрать рабочий интерфейс, а ИИ поможет пройти путь от нуля.

Требования

- Фронтенд реализуется как отдельная часть приложения.
- Фронтенд получает данные и выполняет действия только через API по контракту.
- Интерфейс должен корректно работать с отдельно запущенным бэкендом.

Ссылки

- [Vite](#) — быстрый dev-сервер и сборка фронтенда.

- [shadcn/ui](#) — набор UI-компонентов с гибкой настройкой.
- [Mantine](#) — готовая UI-библиотека для быстрого старта.
- [Prism](#) — эмулятор API по контракту для разработки и проверки фронтенда.
- [shadcn MCP Server](#) — MCP сервер позволяет агентам взаимодействовать с элементами shadn.

Задачи

1. Выберите, на чем будет собран фронтенд.
2. Реализуйте страницы интерфейса.
3. Подключите интерфейс к API по контракту.

Результат

Появился UI приложения.

4. Реализация бэкенда

На этом шаге мы реализуем серверную часть приложения по контракту, который зафиксировали ранее.

Рекомендуем выбрать знакомый и простой стек бэкенда: можно использовать любые технологии, с которыми вам удобнее.

Реализация должна идти от контракта, а не от выбранного фреймворка.

Требования

- Бэкенд предоставляет API по спецификации контракта из шага проектирования.
- API бэкенда предназначен для отдельного фронтенд-клиента.
- Основные бизнес-правила бронирования реализуются на стороне бэкенда.
- Для этого шага отдельная база данных не нужна. Достаточно простого хранилища в памяти: после перезапуска сервиса данные могут сбрасываться.

Задачи

1. Выберите технологии, которые будете использовать для бэкенда.
2. Реализуйте API по созданной спецификации контракта.
3. Сделайте обработку ситуации, когда выбранный слот уже занят.

Результат

Есть рабочий бэкенд с хранением данных.

5. Интеграционные сценарии (Playwright)

На этом шаге мы проверяем приложение на уровне пользовательских сценариев. Здесь важно убедиться, что фронтенд и бэкенд работают вместе и что ключевой путь бронирования действительно проходит от начала до конца.

Рекомендуем использовать Playwright. Playwright — это инструмент e2e-тестирования: он управляет реальным браузером, повторяет действия пользователя и проверяет ожидаемый результат на экране. С его помощью мы можем протестировать пользовательские сценарии приложения. Язык реализации тестов не принципиален, но с типизированными языками агенты работают лучше.

Если вам удобнее другой e2e-инструмент, можно использовать его. Важно, чтобы тесты покрывали требования шага и проверяли сценарии в реальном браузере.

Кроме автоматического запуска тестов, в эту же CI-обязку добавим автоматизацию релизов. Для этого договоримся о формате сообщений коммитов по спецификации Conventional Commits (`feat:`, `fix:` и так далее) и подключим release-please. Этот инструмент анализирует историю коммитов и сам поддерживает release-PR: формирует changelog и предлагает новую версию по правилам семантического версионирования. При агентной разработке это особенно полезно: вы просите агента писать коммиты в нужном формате, а вся работа с версиями и историей изменений выполняется без ручного труда.

Требования

Интеграционные тесты должны покрывать основной сценарий бронирования.

Ссылки

- [Playwright MCP](#) — запуск браузерных сценариев и автоматизация пользовательских действий через агента.
- [Chrome Devtools MCP](#) — диагностика проблем в браузере: сеть, консоль, состояние страницы.
- [Conventional Commits](#) — спецификация формата сообщений коммитов.
- [release-please](#) — генерация release-PR с changelog и версией на основе Conventional Commits.
- [release-please-action](#) — готовый GitHub Action для подключения release-please.

Задачи

1. Опишите и зафиксируйте основные пользовательские сценарии для проверки.
2. Подготовьте интеграционные тесты.
3. Настройте запуск тестов в CI, например через GitHub Actions.
4. Договоритесь о формате коммитов по Conventional Commits, в том числе для коммитов, которые делает агент.
5. Подключите release-please как GitHub Actions workflow.
6. Убедитесь, что после мёрджа в основную ветку release-please создаёт или обновляет release-PR с changelog и предложенной версией.

Результат

Есть автоматические интеграционные проверки, которые покрывают основной сценарий бронирования, а релизы и changelog формируются автоматически по коммитам через release-please.

6. Docker и деплой

На этом шаге мы готовим приложение к запуску в продакшене. Сделаем это с помощью Docker. Он позволяет упаковать приложение и его окружение в один образ, который одинаково запускается локально, в CI и в облаке. Это удобный способ развертывания: меньше различий между средами и проще повторить запуск на любой платформе.

По итогам шага должен получиться будет описанный Docker-образ, при запуске которого приложение автоматически стартует внутри контейнера.

Требования

- В репозитории должен быть файл *Dockerfile*. По нему в проверке собирается образ и запускается приложение.
- При запуске контейнера из собранного образа приложение должно стартовать автоматически.
- Приложение должно запускаться в контейнере по порту из переменной окружения `PORT`. Эта переменная используется и при деплое, и в автоматической проверке проекта.
- После деплоя у приложения должна быть публичная ссылка.

Ссылки

- [Render MCP](#) — нужен, чтобы агент мог автоматизировать деплой и проверку сервиса в Render.
- [Гайд Как деплоить приложение на Render](#)

Задачи

1. Соберите Docker-образ приложения.
2. Настройте деплой, например на Render, с запуском приложения по `PORT`. Проверьте работу приложения.
3. Добавьте в репозиторий ссылку на опубликованное приложение.

Результат

Есть *Dockerfile* для сборки образа, приложение запускается в контейнере по `PORT`.

7. Как мы проверяем проект

- Формально проект проходит простую проверку: сборку и запуск приложения в контейнере.
- Главный целевой критерий: в идеале весь код проекта написан с помощью агента.
- Статус проверки можно посмотреть в репозитории GitHub.
- Вывод проверки находится во вкладке *Actions* в workflow `hexlet-check`.
- Подробнее об автоматической проверке можно почитать [здесь](#).

Что должно работать

В проекте должны корректно работать основные пользовательские сценарии:

- **Бронирование слота:** пользователь может пройти путь записи от выбора слота до успешного подтверждения, а созданная запись сохраняется и отображается в интерфейсе.
- **Занятость слотов:** если слот уже забронирован, повторная запись на него не должна проходить; пользователь получает понятное сообщение о конфликте.

Требования к оформлению

- Проект выполнен в подходе **Design First**: фронтенд и бэкэнд реализованы отдельно и взаимодействуют через API-контракт.
- В репозитории есть *Dockerfile* для запуска приложения.
- Приложение успешно собирается и запускается в Docker.
- Приложение запускается на порту, который передается через переменную окружения `PORT`.

Что дальше

После успешной проверки можно развивать проект как продукт и добавлять новые возможности.

- **Кастомное расписание:** гибкие окна доступности по дням недели, перерывы, исключения и праздничные дни.
 - **Таймзоны:** хранение таймзоны владельца календаря и корректное отображение слотов для гостей из других регионов.
 - **Регистрация и аккаунты:** поддержка нескольких пользователей и нескольких календарей/типов встреч у каждого пользователя.
 - **Интеграции с календарями:** синхронизация занятости и событий с внешними календарными сервисами.
 - **Уведомления:** напоминания о предстоящих событиях (email, Telegram, push), подтверждения и отмены.
 - **Дополнительные сценарии:** перенос/отмена бронирований, повторяющиеся события, буфер между встречами, аналитика по записям.
-